

Smenul lui Mars – Difference Array (Vector de diferente)

1. Ideea pe scurt

Difference Array (vector de diferente) este o tehnica prin care aplici rapid modificari pe intervale intr-un vector (de exemplu: aduna $+x$ pe toate pozitiile din $[l..r]$) folosind doar $O(1)$ operatii per modificare, urmand ca la final sa reconstruiesi vectorul printr-o suma cumulativa (prefix).

2. Definitie

Fie vectorul $a[0..n-1]$. Definim vectorul de diferente d astfel: $d[0] = a[0]$ $d[i] = a[i] - a[i-1]$, pentru $i \geq 1$. Atunci a se reface din d prin: $a[0] = d[0]$ $a[i] = a[i-1] + d[i]$ (suma prefix pe d).

3. Exemplu: construirea vectorului de diferente

$a = [5, 5, 7, 10]$ $d[0] = 5$ $d[1] = 5 - 5 = 0$ $d[2] = 7 - 5 = 2$ $d[3] = 10 - 7 = 3$ Deci $d = [5, 0, 2, 3]$

4. Exemplu: reconstructie (prefix pe d)

$d = [5, 0, 2, 3]$ $a[0] = 5$ $a[1] = 5 + 0 = 5$ $a[2] = 5 + 2 = 7$ $a[3] = 7 + 3 = 10$ Deci $a = [5, 5, 7, 10]$

5. Update pe interval: $a[l..r] += x$

Trucul: daca vrei sa aduni x pe toate elementele din intervalul $[l..r]$, nu modifici direct a . Modifici doar d astfel: $d[l] += x$ $d[r+1] -= x$ (daca $r+1$ este in interiorul $[0..n-1]$) Apoi, dupa ce ai aplicat toate update-urile, refaci a prin suma prefix pe d .

6. Exemplu complet (update-uri multiple)

Pornim cu $a = [0, 0, 0, 0, 0]$ ($n=5$). Initial $d = [0, 0, 0, 0, 0]$. Update 1: aduna $+3$ pe $[1..3]$ $d[1] += 3$, $d[4] -= 3 \Rightarrow d = [0, 3, 0, 0, -3]$ Update 2: aduna $+2$ pe $[0..2]$ $d[0] += 2$, $d[3] -= 2 \Rightarrow d = [2, 3, 0, -2, -3]$ Reconstructie (prefix): $a[0] = 2$ $a[1] = 2 + 3 = 5$ $a[2] = 5 + 0 = 5$ $a[3] = 5 + (-2) = 3$ $a[4] = 3 + (-3) = 0$ Rezultat: $a = [2, 5, 5, 3, 0]$

7. De ce merge (intuire)

d controleaza 'salturile' dintre valori consecutive. Cand faci $d[l] += x$, practic spui: de la pozitia l incolo, toate valorile cresc cu x . Cand faci $d[r+1] -= x$, anulezi cresterea incepand cu pozitia $r+1$. Suma prefix transforma aceste doua marcaje intr-o crestere constanta pe interval.

8. Cand se foloseste (tipic BAC/contest)

- multe operatii de tip: adauga x pe interval - marcare de intervale (ex: drumuri, intervale de timp, acoperiri) - construirea unui vector final dupa aplicarea mai multor 'reguli' pe intervale

9. Implementare minimala (fara STL, accent pe idee)

```
#include <iostream>
using namespace std;
```

```
int main(){
```

```

int n, q; // n = dimensiune, q = numar update-uri
cin >> n >> q;
long long d[100000];
for(int i=0;i<n;i++) d[i]=0;

for(int t=0;t<q;t++){
    int l, r; long long x;
    cin >> l >> r >> x; // presupunem 0<=l<=r<n
    d[l] += x;
    if(r+1 < n) d[r+1] -= x;
}

long long cur = 0;
for(int i=0;i<n;i++){
    cur += d[i];
    cout << cur << ' ';
}
return 0;
}

```

10. Cazuri limita

- l=0: doar $d[0] += x$ - r=n-1: nu mai exista $d[r+1]$, deci nu scazi - update-uri cu x negativ (scaderi) sunt ok - atentie la tipul de date: foloseste long long daca sumele pot fi mari

11. Greseli frecvente

- folosirea indicilor 1..n in loc de 0..n-1 fara adaptare - uitarea lui $d[r+1] -= x$ (cand $r+1 < n$) - reconstructie gresita (trebuie prefix: $cur += d[i]$) - vector d neinitializat cu 0

12. Checklist de BAC

- stii definitia $d[i] = a[i] - a[i-1]$ - stii reconstructia prin suma prefix - stii update-ul $O(1)$: $d[l] += x$, $d[r+1] -= x$ - poti explica un exemplu numeric cap-coada